Custom SOC SIEM Pipeline with ELK and Sigma Rules

This project demonstrates building a **full SOC-level log ingestion, detection, and alerting pipeline** using **ELK Stack** (Elasticsearch, Logstash, Kibana) and **Sigma rules**.

Table of Contents

1. Project Overview
2. Prerequisites installation
3. Step 1 — Install Docker
4. Step 2 — Deploy ELK Stack
5. Step 3 — Add Logstash for Log Ingestion
6. Step 4 — Connect Host Logs to Logstash
7. Step 5 — Implement Sigma Detection Rules
8. Step 6 — Configure Automated Alerts in Kibana
9. Dashboard

Project Overview

This project simulates a **Security Information and Event Management (SIEM)** environment:

- Logs (e.g., login attempts) are ingested via **Logstash**.
- Logs are stored and indexed in **Elasticsearch**.
- **Kibana** visualizes logs and alerts.
- **Sigma rules** detect suspicious activity and automate alerts.

Analogy: Logs = CCTV footage, ELK = storage + dashboard, Sigma = security guard detecting threats.

Prerequisites

- Kali Linux or Linux with Docker support
- Python 3 for Sigma tools
- Basic command knowledge

Step 1 — Install Docker

Docker allows running all ELK components as containers.

1.1 Update system packages

```
sudo apt update && sudo apt upgrade -y
```

1.2 Install Docker

```
sudo apt install docker.io -y
```

1.3 Start Docker service

```
sudo systemctl start docker && sudo systemctl enable docker
```

1.4 Add user to Docker group

```
sudo usermod -aG docker $USER && newgrp docker
```

1.5 Verify installation

```
docker --version && docker run hello-world
```

Step 2 — Deploy ELK Stack

2.1 Create project folder

```
mkdir elk_project && cd elk_project
```

2.2 Create `docker-compose.yml`

```
version: '3.8'

services:

  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.
12.0
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
    ports:
      - "9200:9200"
    networks:
      - elk

  kibana:
    image: docker.elastic.co/kibana/kibana:8.12.0
    container_name: kibana
    ports:
      - "5601:5601"
    depends_on:
      - elasticsearch
    networks:
      - elk

networks:
  elk:
```

2.3 Start ELK Stack

```
docker-compose up -d docker ps
```

2.4 Open Kibana

```
http://localhost:5601
```

Step 3 — Add Logstash for Log Ingestion

3.1 Edit `docker-compose.yml` to add Logstash

```
  logstash:
    image: docker.elastic.co/logstash/logstash:8.12.0
    container_name: logstash
    volumes:
      - ./logstash_pipeline:/usr/share/logstash/pipeline
    ports:
      - "5044:5044"
    depends_on:
      - elasticsearch
    networks:
      - elk
```

3.2 Create Logstash pipeline folder

```
mkdir logstash_pipeline
cd logstash_pipeline
nano logstash.conf
```

3.3 Logstash configuration (`logstash.conf`)

```
input {
  file {
    path => "/var/log/auth.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }
}

filter {
  grok {
    match => { "message" => "%{SYSLOGTIMESTAMP:timestamp} %
{HOSTNAME:host} %{DATA:process}: %{GREEDYDATA:log_message}"
}
  }
}
```

```
output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "auth-logs"
  }
}
```

3.3 Restart containers

```
docker-compose down
docker-compose up -d
docker ps
```

Step 4 — Connect Host Logs to Logstash

4.1 Modify Logstash volumes in `docker-compose.yml`

```
    volumes:
      - ./logstash_pipeline:/usr/share/logstash/pipeline
      - /var/log/auth.log:/var/log/auth.log
```

4.2 Restart containers

```
docker-compose down
docker-compose up -d
```

4.3 Create Kibana index pattern

- Stack Management → Index Patterns → Create index pattern
- Enter: `auth-logs*`
- Time field: `@timestamp`

4.4 Verify logs

- Discover → select `auth-logs*`
- Logs like SSH login attempts should appear

Step 5 — Implement Sigma Detection Rules

5.1 Install Sigma conversion tool

```
pip install sigmatools
```

5.2 Create Sigma folder

```
mkdir ~/sigma_rules
cd ~/sigma_rules
```

5.3 Create detection rule (`ssh_bruteforce.yml`)

```
title: SSH Brute Force Detection
id: 12345
status: experimental
description: Detect multiple failed SSH login attempts
author: beginner-soc
logsource:
  product: linux
  service: sshd

detection:
  selection:
    log_message: "Failed password"
  condition: selection

level: high
```

5.4 Convert Sigma to Elasticsearch query

```
sigmac -t es-qs ssh_bruteforce.yml
# Output: log_message:"Failed password"
```

5.5 Test in Kibana Discover

- Search bar: `log_message:"Failed password"`
- Simulate failed SSH logins:

```
ssh fakeuser@localhost
```

Step 6 — Configure Automated Alerts in Kibana

1.Go to Security → Rules → Create Rule

2.Select Rule Type: Custom Query Rule

3.Configure detection

- Index: `auth-logs*`
- Query: `log_message:"Failed password"`
- Time field: `@timestamp`
- Schedule: Every 1 minute, Look back 5 minutes
- Trigger: More than 5 matches

4.Name and enable rule: `SSH Brute Force Detection`

5.Simulate attack with multiple failed logins

6.Check Security → Alerts for triggered alerts

The Project has been completed successfully

Done By

RUTHRAN-SEC